

Motion Characterization Pipeline

Step One: Input Data

Step One: Input Data

```
graph TD; A[Step One: Input Data] --> B[Input fMRI Data as either:]; B --> C[Single NiFTI File and Single JSON File]; B --> D[Single DICOM Directory];
```

Input fMRI Data as either:

Single NiFTI File and
Single JSON File

Single DICOM Directory

Step One: Input Data

Input fMRI Data as either:

Single DICOM Directory

Use docker image
crl/dicom-tools to do
dcm2niix

```
self.run_command(  
    command=[  
        "docker", "run", "--rm",  
        "-v", f"{self.working_directory}/data"  
        "crl/dicom-tools", "dcm2niix",  
        "-o", ".",  
        "-z", "y",  
        "-b", "y",  
        "-ba", "n",  
        "-w", "1",  
        os.path.basename(dicom_directory)  
    ],  
    verbose=True  
)
```

Step One: Input Data

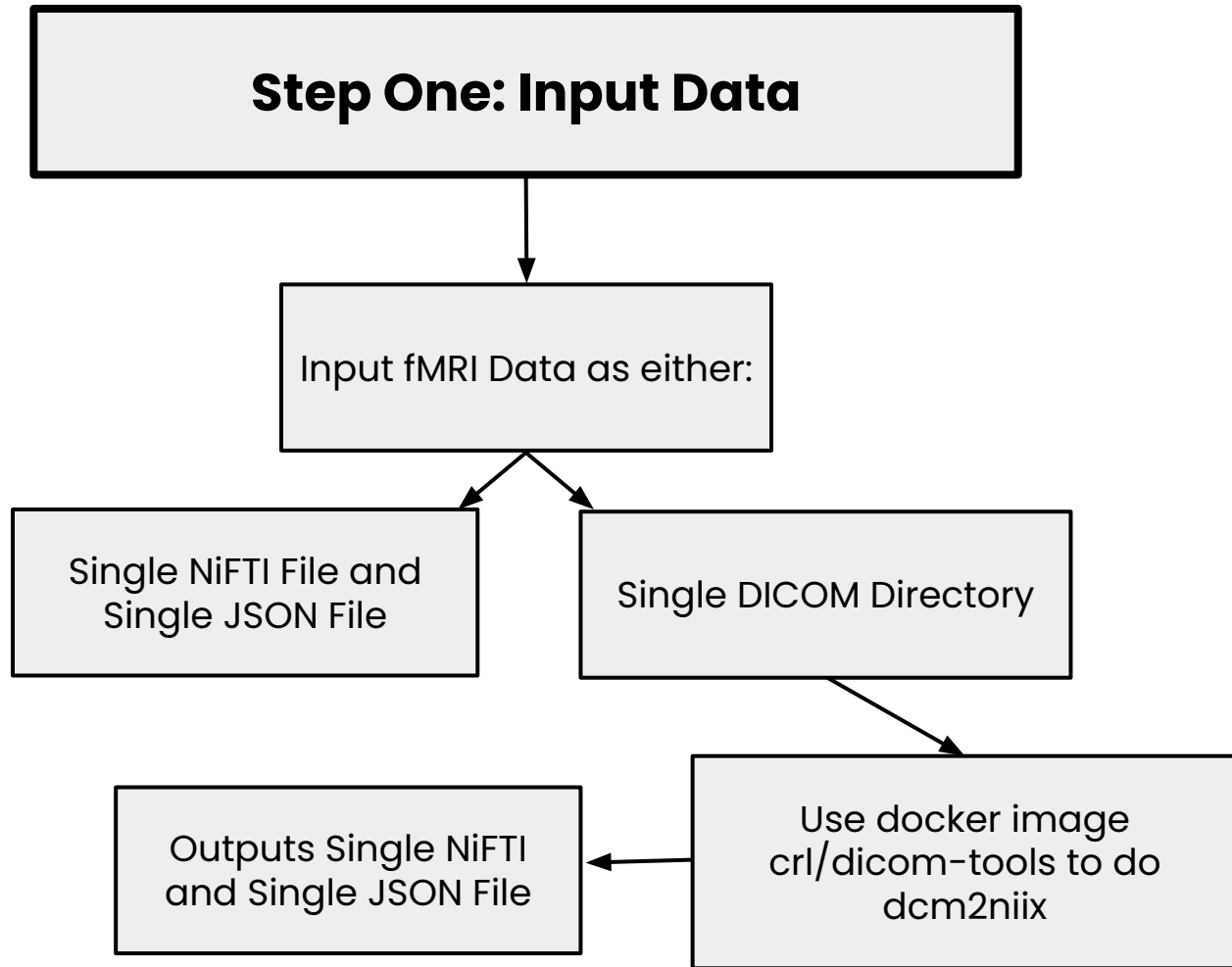
Input fMRI Data as either:

Single NiFTI File and
Single JSON File

Single DICOM Directory

Outputs Single NiFTI
and Single JSON File

Use docker image
crl/dicom-tools to do
dcm2niix



Step Two: Get Slice Timing

Step Two: Get Slice Timing

This pipeline is for SMS-Accelerated fMRI Data. We need to determine which slices are acquired at the same time.

Step Two: Get Slice Timing

Load JSON File as
Dictionary

```
def get_slice_timing(self, json_path):  
  
    def find_matching_indexes(numbers):  
  
        num_index_map = {}  
  
        for index, number in enumerate(numbers):  
            if number in num_index_map:  
                num_index_map[number].append(index)  
            else:  
                num_index_map[number] = [index]  
  
        return {number: indexes for number, indexes in num_index_map.items() if len(indexes) > 1}  
  
    with open(json_path) as f:  
        json_data = json.load(f)  
        if not 'SliceTiming' in json_data:  
            print(f"'SliceTiming' Key Not In JSON File.")  
            exit(0)  
        else:  
            return OrderedDict(sorted(find_matching_indexes(json_data['SliceTiming']).items()))
```

Step Two: Get Slice Timing

Load JSON File as
Dictionary

Extract 'SliceTiming' Key

```
def get_slice_timing(self, json_path):  
  
    def find_matching_indexes(numbers):  
  
        num_index_map = {}  
  
        for index, number in enumerate(numbers):  
            if number in num_index_map:  
                num_index_map[number].append(index)  
            else:  
                num_index_map[number] = [index]  
  
        return {number: indexes for number, indexes in num_index_map.items() if len(indexes) > 1}  
  
    with open(json_path) as f:  
        json_data = json.load(f)  
        if not 'SliceTiming' in json_data:  
            print(f"'SliceTiming' Key Not In JSON File.")  
            exit(0)  
        else:  
            return OrderedDict(sorted(find_matching_indexes(json_data['SliceTiming'].items())))
```

Step Two: Get Slice Timing

```
def get_slice_timing(self, json_path):  
    def find_matching_indexes(numbers):  
        num_index_map = {}  
  
        for index, number in enumerate(numbers):  
            if number in num_index_map:  
                num_index_map[number].append(index)  
            else:  
                num_index_map[number] = [index]  
  
        return {number: indexes for number, indexes in num_index_map.items() if len(indexes) > 1}  
  
    with open(json_path) as f:  
        json_data = json.load(f)  
        if not 'SliceTiming' in json_data:  
            print(f"'SliceTiming' Key Not In JSON File.")  
            exit(0)  
        else:  
            return OrderedDict(sorted((find_matching_indexes(json_data['SliceTiming']).items())))
```

Load JSON File as
Dictionary

Extract 'SliceTiming' Key

Sort Into Dictionary

```
Slice Timing Information:
Slice Group 1 of 15:
  Aquisition Time: 0s
  Slices in Group: [0, 15, 30, 45]
Slice Group 2 of 15:
  Aquisition Time: 0.07s
  Slices in Group: [2, 17, 32, 47]
Slice Group 3 of 15:
  Aquisition Time: 0.14s
  Slices in Group: [4, 19, 34, 49]
Slice Group 4 of 15:
  Aquisition Time: 0.21s
  Slices in Group: [6, 21, 36, 51]
Slice Group 5 of 15:
  Aquisition Time: 0.2775s
  Slices in Group: [8, 23, 38, 53]
Slice Group 6 of 15:
  Aquisition Time: 0.3475s
  Slices in Group: [10, 25, 40, 55]
Slice Group 7 of 15:
  Aquisition Time: 0.4175s
  Slices in Group: [12, 27, 42, 57]
Slice Group 8 of 15:
  Aquisition Time: 0.4875s
  Slices in Group: [14, 29, 44, 59]
Slice Group 9 of 15:
  Aquisition Time: 0.555s
  Slices in Group: [1, 16, 31, 46]
Slice Group 10 of 15:
  Aquisition Time: 0.625s
  Slices in Group: [3, 18, 33, 48]
Slice Group 11 of 15:
  Aquisition Time: 0.695s
  Slices in Group: [5, 20, 35, 50]
Slice Group 12 of 15:
  Aquisition Time: 0.7625s
  Slices in Group: [7, 22, 37, 52]
Slice Group 13 of 15:
  Aquisition Time: 0.8325s
  Slices in Group: [9, 24, 39, 54]
Slice Group 14 of 15:
  Aquisition Time: 0.9025s
  Slices in Group: [11, 26, 41, 56]
Slice Group 15 of 15:
  Aquisition Time: 0.9725s
  Slices in Group: [13, 28, 43, 58]
```

et Slice Timing

Load JSON File as
Dictionary

Extract 'SliceTiming' Key

Sort Into Dictionary

Step Three: Cut Up Time Series

Step Three: Cut Up Time Series

Cut 4d Time Series Into
3d Volumes using
Docker Image
crl/sms-mi-reg

```
def extract_volumes(self, nifti_image_path):  
  
    self.run_command(  
        command=[  
            "docker", "run", "--rm",  
            "-v", f"{self.working_directory}:/data",  
            "crl/sms-mi-reg", "crl-extract-volumes-from-fmri.py",  
            os.path.basename(nifti_image_path),  
            "volume_outputs.nii"  
        ],  
        verbose=False  
    )  
  
    return sorted(glob(os.path.join(self.working_directory, "volume_outputs-*.nii")))
```

Step Three: Cut Up Time Series

Cut 4d Time Series Into
3d Volumes using
Docker Image
crl/sms-mi-reg

Cut Each 3d Volume into
2d Slices using Docker
Image crl/sms-mi-reg

```
def extract_slices_from_volume(self, volume_paths, n_jobs = 1):  
  
    Parallel(n_jobs=n_jobs)(  
        delayed(self.run_command)(  
            command=[  
                "docker", "run", "--rm",  
                "-v", f"{self.working_directory}:/data",  
                "crl/sms-mi-reg", "crl-fmri-volume-to-slices.py",  
                os.path.basename(volume_path),  
                f"slice_outputs-{ '{:04d}'.format(volume_num)}.nii"  
            ],  
            verbose=False  
        )  
        for volume_num, volume_path in enumerate(volume_paths))  
  
    return sorted(glob(os.path.join(self.working_directory, "slice_outputs-*.nii")))
```

Cut each 3d Volume in Parallel (1 per each CPU
Core) using Joblib.Parallel and Joblib.delayed
unless 'Serialize' Argument is Given

Step Four: Upsample Reference Volume

Step Four: Upsample Reference Volume

We found upsampling the reference volume
better the alignment.

By default, upsampling is set to True and
dimensions we upsample to are:
1.236mm X 1.236mm X 1.236mm

Step Four: Upsample Reference Volume

```
# Get reference volume from index in the list of created 3D volumes
reference_volume_path = volume_paths[reference_volume_index]
if upsample_reference_volume:
    reference_volume_path = self.upsample_reference_volume(
        reference_volume_path,
        spacing=reference_volume_spacing
    )
print(f"Reference Volume Path: {reference_volume_path}")
```

```
def upsample_reference_volume(self, reference_volume_path, spacing):
    def resample_img(img, spacing, sz, interpolator = sitk.sitkLinear):
        # interpolator could be sitk.sitkLinear
        # interpolator could be sitk.sitkBSpline
        r = sitk.ResampleImageFilter()
        r.SetInterpolator(interpolator)
        r.SetOutputPixelType( img.GetPixelID() )
        r.SetDefaultPixelValue(0)
        r.SetOutputOrigin(img.GetOrigin())
        r.SetOutputSpacing(spacing)
        r.SetOutputDirection(img.GetDirection())
        r.SetSize(sz)
        return r.Execute(img)

    def resample_img_new_spacing(img, new_spacing):
        spacing = np.array(img.GetSpacing())
        sz = np.array(img.GetSize())
        new_sz = np.floor(spacing / new_spacing * sz).astype(np.uint32)
        new_sz = 2*np.floor((new_sz+1)/2).astype(np.uint32)
        print(f"Reference Volume New Size: {new_sz}")
        return resample_img(img, new_spacing, new_sz.tolist())

    upsampled_volume = resample_img_new_spacing(
        sitk.ReadImage(reference_volume_path),
        new_spacing=spacing
    )
    output_volume_path = os.path.join(
        self.working_directory,
        f"upsampled_{os.path.basename(reference_volume_path)}"
    )
    sitk.WriteImage(
        upsampled_volume,
        output_volume_path
    )

    return output_volume_path
```

Step Five: Create Identity Transform

Step Five: Create Identity Transform



The identity transform
performs two roles:

Step Five: Create Identity Transform

```
graph TD; A[Step Five: Create Identity Transform] --> B[The identity transform performs two roles:]; B --> C[1. Initialize the first slice-group alignment];
```

The identity transform
performs two roles:

1. Initialize the first
slice-group
alignment

Step Five: Create Identity Transform

```
graph TD; A[Step Five: Create Identity Transform] --> B[The identity transform performs two roles:]; B --> C[1. Initialize the first slice-group alignment]; B --> D[2. Identify the rotation center of the reference volume];
```

The identity transform
performs two roles:

1. Initialize the first
slice-group
alignment

2. Identify the
rotation center of
the reference
volume

Step Five: Create Identity Transform



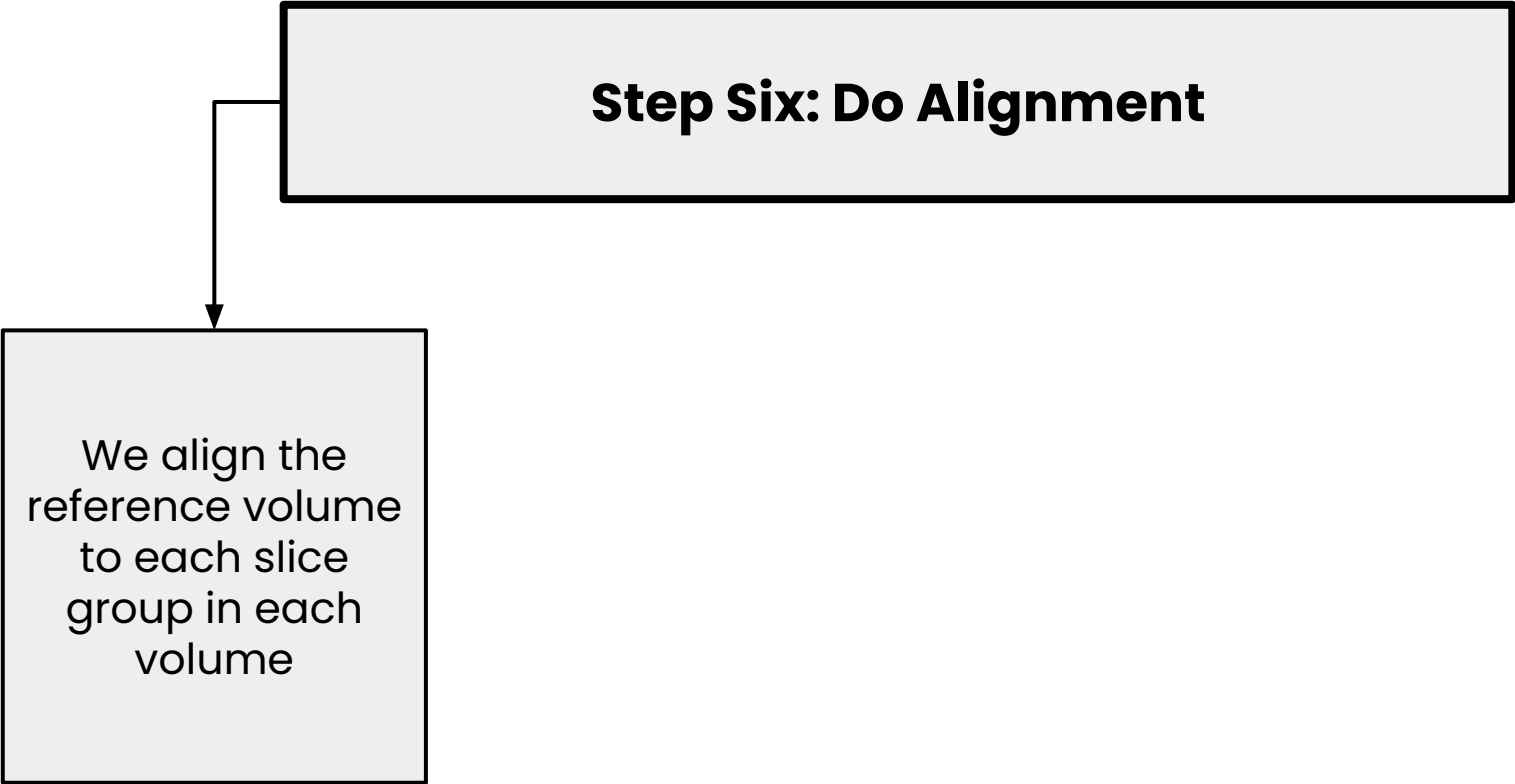
```
def make_identity_transform(self, reference_volume_path):
    print(f"Creating Identity Transform at the Volume Center of {os.path.basename(reference_volume_path)}")

    self.run_command(
        command=[
            "docker", "run", "--rm",
            "-v", f"{self.working_directory}:/data",
            "crl/sms-mi-reg", "crl-identity-transform-at-volume-center.py",
            "--refvolume", os.path.basename(reference_volume_path),
            "--transformfile", "identity-centered.tfm"
        ],
        verbose=True
    )

    return os.path.join(self.working_directory, "identity-centered.tfm")
```

Step Six: Do Alignment

Step Six: Do Alignment



```
graph TD; A[Step Six: Do Alignment] --> B[We align the reference volume to each slice group in each volume];
```

We align the
reference volume
to each slice
group in each
volume

Step Six: Do Alignment

We align the
reference volume
to each slice
group in each
volume

```
def do_alignments_at_one_volume(reference_volume_path, volume_path, volume_num,
                                slice_timing, transform_paths, initialization_method,
                                slice_level_n_jobs):

    if initialization_method == 'last_volume':
        output_volume_level_transform = self.do_volume_level_alignment(
            reference_volume_path=reference_volume_path,
            volume_path=volume_path,
            volume_num=volume_num,
            transform_paths=transform_paths
        )
        transform_paths.append(output_volume_level_transform)

    Parallel(n_jobs=slice_level_n_jobs)(
        delayed(self.do_slice_group_level_alignment)(
            reference_volume_path=reference_volume_path,
            initial_transform=transform_paths[-1],
            transform_paths=transform_paths,
            volume_num=volume_num,
            slice_group_num=slice_group_num,
            slice_list_indexes=slice_list_indexes,
            initialization_method=initialization_method
        )
        for slice_group_num, slice_list_indexes in enumerate(slice_timing.values())
    )
```

Step Six: Do Alignment



You can initialize the
volume-to-slice-group
alignment in one of three
ways:

Step Six: Do Alignment



You can initialize the volume-to-slice-group alignment in one of three ways:

Initialize with Identity Transform

Step Six: Do Alignment

You can initialize the
volume-to-slice-group
alignment in one of three
ways:

Initialize with Identity
Transform

Pros:

This is the fastest
option: Parallelize at
the volume level
w/out having to run
volume-to-volume
alignment

Cons:

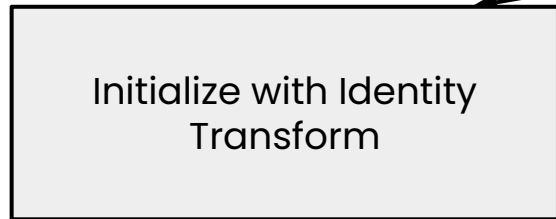
Optimizer may be
initialized from a
starting point that
too far away. As a
result, this could be
the least accurate
option.

Step Six: Do Alignment

You can initialize the
volume-to-slice-group
alignment in one of three
ways:

Initialize with Identity
Transform

Initialize with
Volume-to-Volume
Transform



Step Six: Do Alignment

You can
volume
alignme

Pros:

Faster than slice-level initialization and more likely to be accurate than the identity transform initialization

Cons:

May not be as accurate as slice-level initialization. Optimizer may still be initialized from a starting point that too far away in some data

Initialize with Identity Transform

Initialize with Volume-to-Volume Transform

The volume-to-volume transform initialization requires us to do a volume-to-volume alignment. We use the tool 'crlRigidRegistration' inside the Docker image 'crl/crkit'

ways:

nmment

```
def do_volume_level_alignment(self, reference_volume_path, volume_path, volume_num, transform_paths):
    print(f"Aligning Reference Volume '{os.path.basename(reference_volume_path)}' to Target Volume" \
          + f"'{os.path.basename(volume_path)}'")

    output_transform_label = f"align-{'{:04d}'.format(volume_num)}"
    output_transform_path = os.path.join(self.working_directory, output_transform_label + ".tfm")

    self.crlRigidRegistration(
        reference_volume_path=reference_volume_path,
        target_volume_path=volume_path,
        input_transform_path=transform_paths[-1],
        output_transform_label=output_transform_label
    )

    if not os.path.exists(output_transform_path):
        print(f"\nERROR: Transform expected to be created after aligning volume {volume_num + 1}" \
              + " was not created: {output_transform_path}")
        exit(0)

    return output_transform_path
```

```
def crlRigidRegistration(self,
                        reference_volume_path,
                        target_volume_path,
                        input_transform_path,
                        output_transform_label):

    # align reference to target volume in order to fit wi
    self.run_command(
        command=[
            "docker", "run", "--rm",
            "-v", f"{self.working_directory}:/data",
            "crl/crkit", "crlRigidRegistration",
            "-l", os.path.basename(input_transform_path),
            os.path.basename(target_volume_path),
            os.path.basename(reference_volume_path),
            output_transform_label + '.nii',
            output_transform_label + '.tfm'
        ],
        verbose=True
    )
```

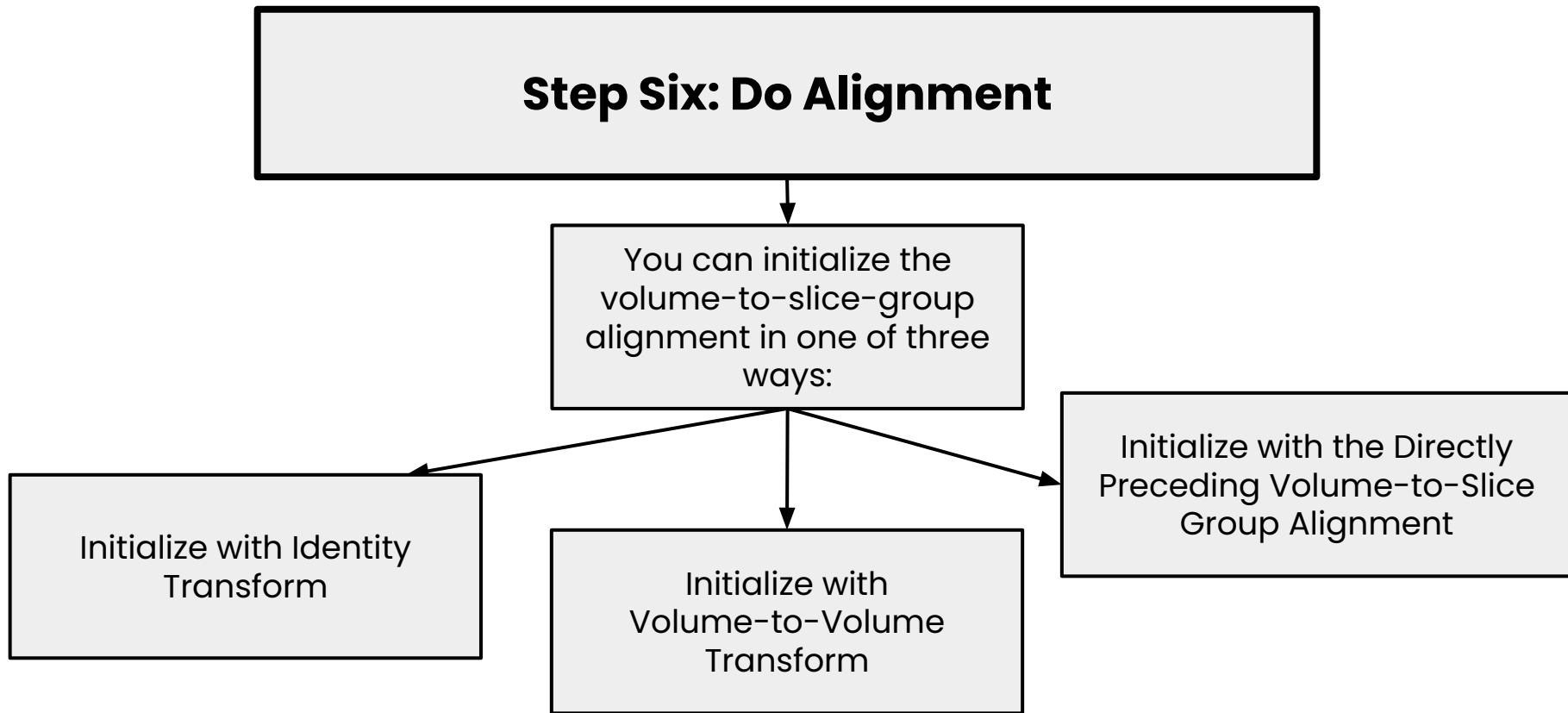
Step Six: Do Alignment

You can initialize the
volume-to-slice-group
alignment in one of three
ways:

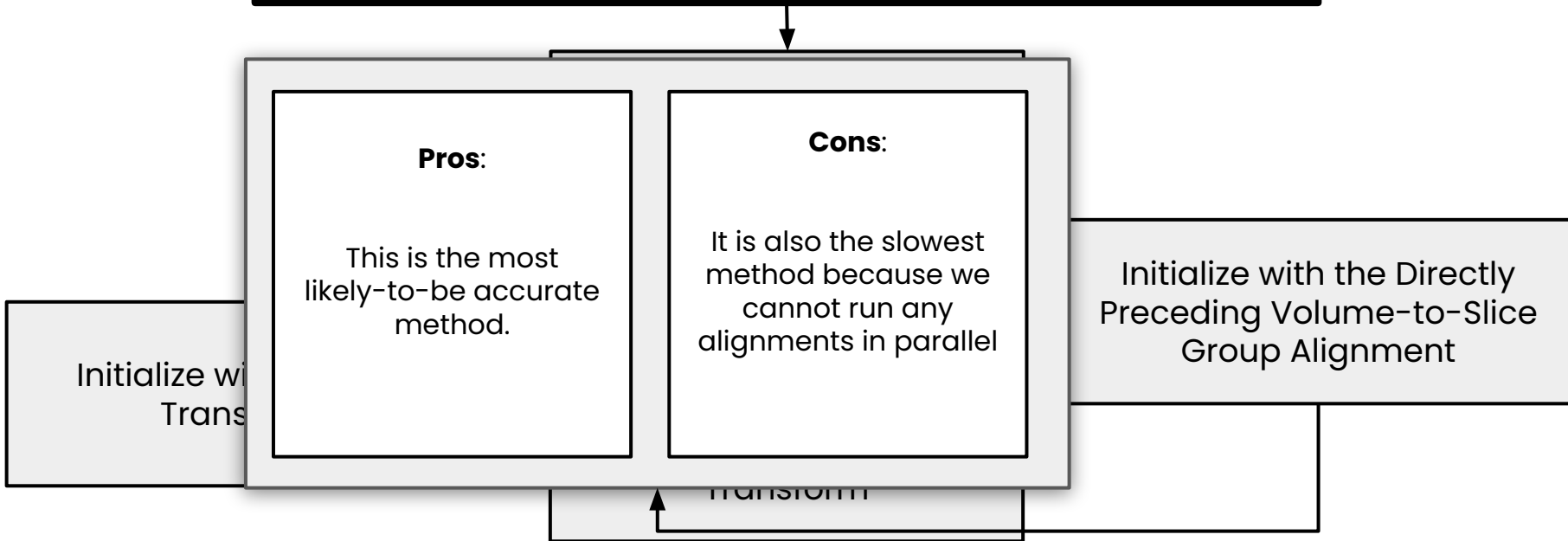
Initialize with Identity
Transform

Initialize with
Volume-to-Volume
Transform

Initialize with the Directly
Preceding Volume-to-Slice
Group Alignment

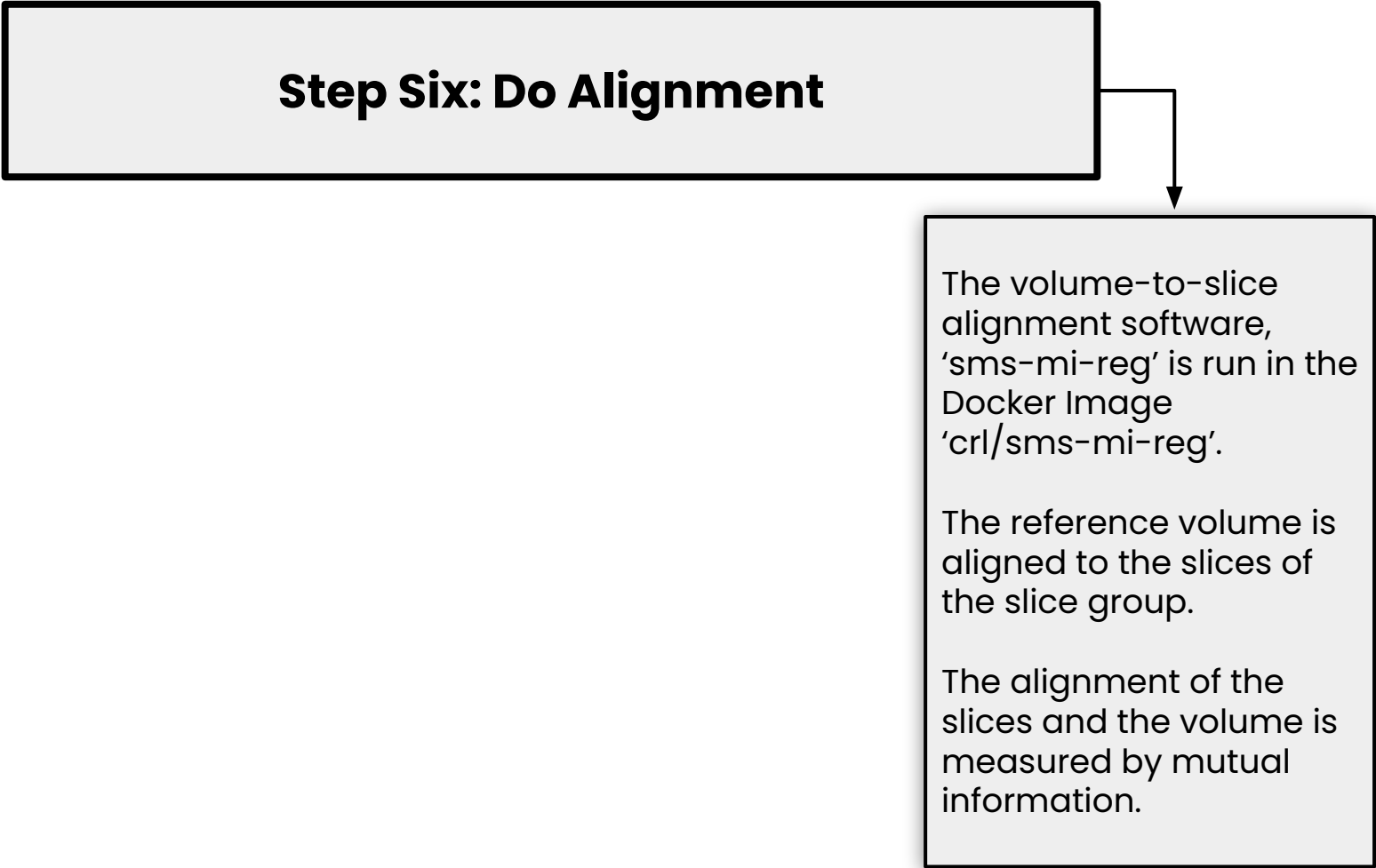


Step Six: Do Alignment



Step Six: Do Alignment

Step Six: Do Alignment



```
graph TD; A[Step Six: Do Alignment] --> B[The volume-to-slice alignment software, 'sms-mi-reg' is run in the Docker Image 'crl/sms-mi-reg'.  
The reference volume is aligned to the slices of the slice group.  
The alignment of the slices and the volume is measured by mutual information.];
```

The volume-to-slice alignment software, 'sms-mi-reg' is run in the Docker Image 'crl/sms-mi-reg'.

The reference volume is aligned to the slices of the slice group.

The alignment of the slices and the volume is measured by mutual information.

Step Six: Do Alignment

```
def sms_mi_reg(self, reference_volume_path, input_transform_path, output_transform_label, input_slice_paths):
    """
    Usage: sms-mi-reg [--help] [--version] [--optimizer VAR] [--maxiter VAR] referenceVolume inputTransform outputTransformLabel inputSlices

    Positional arguments:
    referenceVolume      The volume that is moving to be aligned to the slices.
    inputTransform        The transform to initialize the alignment.
    outputTransformLabel  Name phrase used in the construction of the output transform file name.
    inputSlices           The list of file names of the fixed target slices. [nargs: 1 or more]

    Optional arguments:
    -h, --help            shows help message and exits
    -v, --version          prints version information and exits
    --optimizer            Choice of optimizer (LN_COBYLA, LN_BOBYQA, LN_NELDERMEAD, LN_SBPLX). Default is LN_SBPLX. [default: "LN_SBPLX"]
    --maxiter              Maximum number of optimizer iterations. Default is 1000. [default: 1000]
    """

    self.run_command(
        command=[
            "docker", "run", "--rm",
            "-v", f"{self.working_directory}/data",
            "crl/sms-mi-reg", "sms-mi-reg",
            os.path.basename(reference_volume_path),
            os.path.basename(input_transform_path),
            output_transform_label
        ] + [os.path.basename(slice_path)
            for slice_path in input_slice_paths
        ],
        verbose=True
    )
```

The volume-to-slice alignment software, 'sms-mi-reg' is run in the Docker Image 'crl/sms-mi-reg'.

The reference volume is aligned to the slices of the slice group.

The alignment of the slices and the volume is measured by mutual information.

Step Seven: Calculate Displacements

Step Seven: Calculate Displacements



```
graph TD; A[Step Seven: Calculate Displacements] --> B[We calculate each the displacement (in mm) between each volume-to-slice-group transform and its directly preceding volume-to-slice-group transform];
```

We calculate each the displacement (in mm) between each volume-to-slice-group transform and its directly preceding volume-to-slice-group transform

Step S

We calculate each the displacement (in mm) between each volume-to-slice-group transform and its directly preceding volume-to-slice-group transform

```
def calculate_displacement(self, transform1_path, transform2_path, radius = 50):

    def compose_transforms(transform1, transform2):
        transform1_inverse = transform1.GetInverse()

        A0 = np.asarray(transform2.GetMatrix()).reshape(3,3)
        c0 = np.asarray(transform2.GetCenter())
        t0 = np.asarray(transform2.GetTranslation())

        A1 = np.asarray(transform1_inverse.GetMatrix()).reshape(3,3)
        c1 = np.asarray(transform1_inverse.GetCenter())
        t1 = np.asarray(transform1_inverse.GetTranslation())

        combined_mat = np.dot(A0,A1)
        combined_center = c1
        combined_translation = np.dot(A0, t1+c1-c0) + t0+c0-c1
        combined_affine = sitk.AffineTransform(combined_mat.flatten(), combined_translation, combined_center)

        return combined_affine

    def convert_affine_to_vorsorrigid(affinetransform):
        vorsorrigid3d = sitk.VorsorRigid3DTransform()
        vorsorrigid3d.SetCenter( affinetransform.GetCenter() )
        vorsorrigid3d.SetTranslation( affinetransform.GetTranslation() )
        vorsorrigid3d.SetMatrix( affinetransform.GetMatrix() )
        return vorsorrigid3d

    transform1 = sitk.ReadTransform(transform1_path)
    transform2 = sitk.ReadTransform(transform2_path)
    composed_affine = compose_transforms(transform1, transform2)
    vorsorrigid3d = convert_affine_to_vorsorrigid(composed_affine)
    parms = np.asarray( vorsorrigid3d.GetParameters() )
    vorsormagsquared = parms[0]*parms[0] + parms[1]*parms[1] + parms[2]*parms[2]
    vorsormag = math.sqrt(vorsormagsquared)
    wsquared = 1 - vorsormagsquared
    w = math.sqrt(wsquared)
    angle = 2.0 * math.atan2( vorsormag, w )
    deltarotationmm = float(radius) * float(angle)
    deltatranslationsquared = abs(parms[3])*abs(parms[3]) + abs(parms[4])*abs(parms[4]) + abs(parms[5])*abs(parms[5])
    deltatranslation = math.sqrt(deltatranslationsquared)
    totalmotion = deltarotationmm + deltatranslation

    return totalmotion
```

This Script Outputs:

1. A Directory of SimpleITK VersorRigid3DTransforms
2. A Text File Containing All Displacement Values